ARBITRAGE FINDER

Setup and Operations Guide

A practical guide to the software delivered in this repository.

Edition: 2 November 2026 · English

Source: docs/setup-and-operations.md
Content fingerprint: 82f80e0b9bf7

Read the support status and limitations before using live odds. A working demo and parser tests do not establish that a bookmaker feed is currently accessible.

Contents

1. What this application does 4

1.1 Read the support status first 4

1.2 The seven English manuals 4

2. Decide what a useful first scan means 5

3. Install on Windows 5

3.1 Get the project and prerequisites 5

3.2 Open the dashboard 6

3.3 Install the optional browser scraper 6

4. Install on macOS or Linux 6

5. Understand the dashboard 6

5.1 Scan controls 6

5.2 Result cards and legs 7

5.3 Source health 7

6. Configure bookmaker scrapers 7

6.1 Minimum configuration process 8

6.2 Regional domains and allowlists 8

7. Add an optional API key in the dashboard 8

8. Run your first complete check 9

8.1 Verify the software in demo mode 9

8.2 Verify one real source at a time 9

9. How the arbitrage calculation works 10

9.1 Rounding matters 10

9.2 Conditions the simple calculation does not cover 10

10. Freshness, matching, and coverage . . . 10

11. Imports and reproducible testing . . . 11

12. Troubleshooting . . . 11

12.1 The startup command is not found . . . 11

12.2 Dependency installation fails . . . 11

12.3 The page does not open . . . 11

12.4 A source is unconfigured . . . 12

12.5 A source is blocked . . . 12

12.6 A source returns zero quotes . . . 12

12.7 The results contain implausible profit . . . 12

12.8 No opportunities appear . . . 12

12.9 Browser mode fails . . . 12

12.10 API key or quota errors . . . 12

13. Maintenance, backups, and updates . . . 13

14. Deployment and zero-cost boundaries . . . 13

15. Add another bookmaker . . . 13

16. Rebuild these PDFs . . . 14

17. Daily operating checklist . . . 14

1. What this application does

Arbitrage Finder is a local, manual scanner for pre-match betting prices. It collects decimal odds from enabled sources, converts them to one quote format, compares equivalent markets across bookmakers, and calculates stake allocations for qualifying opportunities. You control the minimum profit percentage in the dashboard and press Scan when you want a new search. There is no subscription built into the application, no scheduled scanning, no alert service, and no automatic bet placement.

The software runs on your computer. Python, the application code, the browser dashboard, the database, and the scraper framework can all be used without a hosting bill. Internet access and a computer are still required for real sources. An optional external API may impose a quota, require registration, change its plan, or charge for extra usage; the application does not remove those limits.

1.1 Read the support status first

The five requested bookmaker profiles are Bet365 at `bet365.com`, Stake at `stake.com`, 1XBet at `al-1xbet.com/en`, Marathonbet at `marathonbet.com/en`, and Bet105 at `bet105.ag`. These are the sites confirmed by the intended user. Their public-page scraper engine is implemented, but their extraction mappings must be configured and checked against real pages. No profile should be treated as a verified live bookmaker integration simply because its name appears in Settings.

The supplied scraper manuals explain how to configure each profile without a login. When a website requires login, denies automated access, presents a challenge, or does not expose usable public prices, that connector remains unavailable. A profile with no extraction mapping is deliberately unconfigured, rather than reporting invented odds.

An optional The Odds API connector accepts a key through the dashboard. Its bookmaker coverage, supported sports, markets, region, free allowance, and update frequency belong to that provider. A provider account does not establish that all five requested bookmakers are available. Start with one selected sport and only the markets and regions you actually need.

1.2 The seven English manuals

1. This guide explains installation, daily use, settings, troubleshooting, and maintenance.
2. Code Explained describes the files, data flow, calculations, safety checks, storage, and extension points.
3. Bet365 Scraper Setup explains the Bet365 profile.
4. Stake Scraper Setup explains the Stake profile.
5. 1XBet Scraper Setup explains the 1XBet profile.
6. Marathonbet Scraper Setup explains the Marathonbet profile.
7. Bet105 Scraper Setup explains the Bet105 profile.

The editable source documents live in `docs/`. Generated PDFs live in `docs/pdf/`. Keep the Markdown and PDFs together when making changes, and regenerate the PDFs after editing the sources.

2. Decide what a useful first scan means

For the first real scan, choose two bookmakers you can access, one sport, one league, one equivalent market, and a small group of future fixtures. Once you can verify those results against the pages, extend coverage. A request for every sport and league can be supported only to the extent that the enabled data sources expose and label them consistently.

Use decimal odds and a single account currency. A price of 2.20 means a successful one-unit stake returns 2.20 units including the original stake. The scanner's displayed ROI is a percentage of the total amount staked across all legs, not the percentage of one winning payout.

Start with these operating choices:

- Pre-match events only.
- Manual bet placement after checking every selection at the bookmaker.
- One currency, such as EUR.
- A small test bankroll in the calculator; no money is transferred by the software.
- A positive minimum profit threshold to allow room for changes between scan and execution.
- A freshness limit and quote-time spread limit that match the source's actual update behavior.

The scanner can legitimately return zero opportunities. That result is different from a failed source. Inspect the source status and quote counts before deciding whether a scan found no arbitrage or collected no usable data.

3. Install on Windows

3.1 Get the project and prerequisites

1. Download `arbitrage-finder-windows.zip` and extract it to a private folder, such as `C:\Projects\Arbitrage-Finder`. On GitHub, the `downloads/` folder holds the source archive and the PDFs-only archive; open the desired ZIP and choose Download raw file. Code then Download ZIP downloads the entire repository instead. Git is optional for running a downloaded archive. These download options become available once the upload to the repository succeeds.
2. Install Python 3.12 or newer if you do not already have the supported runtime, then install the free `uv` Python package manager using the current instructions at the uv installation documentation. On Windows, the Microsoft package manager is another supported installation route when it is available.
3. Open a new PowerShell window after installing tools so it sees the updated PATH.
4. Change into the project directory. Run all project commands from the directory containing `pyproject.toml` and `uv.lock`.

```
cd C:\Projects\Arbitrage-Finder
uv --version
uv sync --frozen
uv run arbscanner
```

If `uv` is installed as a Python module and is not on PATH, use `py -3.12 -m uv` in place of `uv`, for example `py -3.12 -m uv sync --frozen`. You can install it using `py -3.12 -m pip install uv`.

`uv sync --frozen` installs the versions in the lockfile. The first installation needs Internet access to download packages. Later starts reuse the local environment. Do not delete or hand-edit `uv.lock` to work around an installation error; diagnose the actual error instead.

3.2 Open the dashboard

Keep the terminal open while using the dashboard. Open `http://127.0.0.1:8000` in your browser. The application binds to the loopback interface so only programs on your computer can reach it. Closing the terminal or pressing Ctrl+C stops it. To choose another local port, use `uv run arbscanner --port 8001`; to ask the app to open your default browser, use `uv run arbscanner --open`.

The dashboard has a scan view and a settings view. First use the clearly labeled demo mode to verify the interface and calculator. Demo quotes are synthetic examples; they are not current bookmaker prices and must never be used to place bets.

3.3 Install the optional browser scraper

Plain HTML and JSON sources do not need a browser automation engine. Install the browser extra only when you have identified a publicly accessible JavaScript-rendered page that needs it.

```
uv sync --frozen --extra browser
uv run playwright install chromium
```

This downloads a local browser and uses more disk space than HTTP-only scraping. Browser automation does not supply an account, access rights, permission to scrape, or a way through an access challenge. A source that needs authentication cannot be made public by selecting browser mode.

4. Install on macOS or Linux

Install Git, uv, and the project's supported Python version using your preferred supported toolchain. From a Terminal window, enter the repository and run the same project commands.

```
cd ~/Projects/Arbitrage-Finder
uv sync --frozen
uv run arbscanner
```

For browser scraping, install the optional extra and Chromium. Some Linux distributions also need Chromium's system libraries. Use Playwright's documented dependency installation instructions for your distribution; do not disable browser sandboxing or TLS verification to suppress errors.

```
uv sync --frozen --extra browser
uv run playwright install chromium
```

If a managed environment restricts network access, allow the actual package download and required source domains through its supported environment settings. A normal browser's access on your home computer does not establish that a remote cloud machine can reach the same site.

5. Understand the dashboard

5.1 Scan controls

Minimum profit is the lowest displayed ROI percentage you want to consider. Enter `1` for a one-percent threshold, not `0.01`. The engine applies the threshold to the resulting stake plan and its worst-case payout, rather than treating every reciprocal-odds sum below one as actionable profit.

Bankroll is the total amount used by the calculator for one opportunity. It is not a bank balance connection and does not reserve funds. The same bankroll can appear in many opportunities; those opportunities are alternatives unless you independently have enough money at every bookmaker to take them all.

Maximum odds age rejects old observations. Quote-time spread rejects comparisons that combine prices observed too far apart. These two settings solve different problems: all quotes can be recent while still being too far apart from one another for a fast-moving market.

Sport and league filters narrow displayed or scanned coverage according to the controls and source configuration. A filter does not create data that a source does not supply. For an API, configure the provider's sport keys and requested markets as well as the dashboard's visible filters.

5.2 Result cards and legs

A result identifies the event, start time, sport, league, market, period, settlement rules, total stake, return percentage, and worst-case profit. Each leg identifies the bookmaker, selection, decimal odds, stake, and observation time. An available source link can help you reopen the public page, but it is not proof that the price is still offered or accepted for the suggested amount.

Check every leg before acting:

1. Confirm that the event, participants, date, and league are correct.
2. Confirm the exact market, line, period, and settlement rules on every bookmaker.
3. Re-enter the current odds if any price changed.
4. Check your actual available balance, minimum stake, maximum stake, and payout restrictions at each bookmaker.
5. Verify that all legs remain placeable before committing money.

The software does not guarantee that a displayed opportunity can be executed. A bookmaker can change prices, reject a stake, suspend a market, limit an account, void a bet, or settle under rules that differ from your assumption. Those execution conditions are separate from the mathematical calculation.

5.3 Source health

An unconfigured source needs a URL, a mapping, or another required setting. A blocked source may have returned a robots restriction, an HTTP denial, a rate limit, a login page, or a challenge. An error means a request or parsing step failed. An OK result means the connector produced its documented result; inspect its quote count, timestamp, and message as well.

Do not treat an empty response as proof that no betting events exist. A page layout change can also produce no matching rows. Compare the configured row selectors with the current public page, and run a small test before re-enabling a source for broader scans.

6. Configure bookmaker scrapers

Each requested bookmaker has a profile in Settings. Keep a source disabled until its configuration is meaningful. There are three supported extraction modes:

- HTML mode fetches a public HTTPS page and extracts prices from configured CSS selectors.

- JSON mode fetches a public HTTPS JSON response and maps response fields to normalized quote fields.
- Browser mode opens a public page in an anonymous browser context, waits for a configured element when needed, and applies the same HTML extraction rules to the rendered document.

Choose the simplest mode that retrieves complete, current, public odds. JSON can be easier to map when a website itself exposes a documented public response. A URL copied from an authenticated session is not a public API. Never paste account cookies, passwords, browser profile directories, authorization headers, or session tokens into a scraper configuration.

6.1 Minimum configuration process

1. Open the exact regional site you normally use in a private browser window without logging in.
2. Confirm that the intended pre-match prices are visible publicly.
3. Check the site's relevant automation and data-use rules, plus robots instructions for the target path.
4. Choose a narrow league or event list and a small initial page set.
5. Inspect event containers, quote rows, participant names, start times, market names, lines, statuses, and odds.
6. Configure an exact mapping and explicit settlement assumptions.
7. Save the source in the dashboard and run one manual scan.
8. Compare the result against the visible page, including a known non-arbitrage market.
9. Enable broader coverage only after the normalized data is correct.

The five companion scraper manuals provide the field-level configuration and bookmaker-specific checks. Their example selectors are for a controlled example page, not assertions about a bookmaker's current HTML. Replace them using the real public page you verified.

6.2 Regional domains and allowlists

The profiles use the user-confirmed domains listed at the start of this guide. If you later use a different regional domain, change the allowed host list to the exact public hostname required by the profile and set the URL accordingly. Only use a domain you have verified. Redirects and browser requests must satisfy the connector's restrictions; adding a domain in the dashboard does not grant outbound network access in a restricted hosting environment.

The application rejects private or local network targets for remote-source URLs. Do not use `localhost`, `127.0.0.1`, private IP addresses, or cloud metadata addresses as bookmaker sources. Public source configuration is not a general web proxy.

7. Add an optional API key in the dashboard

No API key is required to run the demo, calculator, imports, or configured public scrapers. If you choose The Odds API, obtain your own account and confirm its current free allowance and supported coverage directly with the provider.

1. Open Settings and find the API-key field.
2. Paste the key into the masked field and press Save.
3. Confirm that the dashboard reports a key is present. It should not reveal the saved key.

4. Use Load available sports for a manual provider catalog request, then copy the chosen exact sport keys into the API source configuration. This operation needs the saved key, is subject to a cooldown, and does not confirm bookmaker coverage. Select the API source's sports, markets, regions, and bookmaker filters conservatively.
5. Run one manual scan and inspect the provider response and any reported remaining quota.
6. Delete the saved key in Settings when you no longer need it.

The key is used by the local backend. Do not put it in URLs you share, screenshots, source code, issue reports, or scraper JSON. The application encrypts the key using Fernet and saves the ciphertext in `.data/scanner.sqlite3`; the local decryption key is `.data/secret.key`. Access to both files can expose the credential. POSIX systems use restrictive file modes; on Windows, protect the folder with your normal current-user file permissions because POSIX modes do not establish a Windows ACL. Keep local application data private and out of version control.

One press of Scan can make several external requests if you select multiple sports or pages. The absence of a scheduler does not make repeated manual scans free of provider limits. If a provider returns a limit or billing error, stop requests and check the account rather than rotating keys or bypassing the restriction.

8. Run your first complete check

8.1 Verify the software in demo mode

1. Start the server.
2. Open the dashboard and select demo mode.
3. Use a modest bankroll and a low positive minimum-profit percentage.
4. Press Scan and wait for completion.
5. Confirm that the result is labeled demo and that each leg has a bookmaker, selection, odds, and stake.
6. Increase the minimum-profit threshold above the sample return and scan again; the opportunity should disappear.
7. Return the threshold to your preferred value.

This verifies the dashboard-to-engine workflow. It does not verify any real bookmaker's data source.

8.2 Verify one real source at a time

1. Switch to live mode.
2. Enable only one configured source.
3. Run a scan and verify quote counts and source status.
4. Open Extracted quote preview under Source health. It shows up to 100 collected canonical quotes even if no opportunity passes. Compare prices, selections, timestamps, and market rules against the actual public page or provider response.
5. Enable a second correctly configured bookmaker for the same fixtures and contracts.
6. Scan again and inspect either a candidate opportunity or a clear no-opportunity result.

Do not lower validation standards just to make the result list nonempty. One badly mapped market can produce many false opportunities. Accurate empty results are more useful than incorrect positive results.

9. How the arbitrage calculation works

For mutually exclusive and exhaustive outcomes with decimal odds `o1`, `o2`, and, when applicable, `o3`, add the reciprocals:

```
S = 1/o1 + 1/o2 [+ 1/o3]
Theoretical ROI = (1/S - 1) * 100 percent
Ideal stake for outcome i = total_stake / (oi * S)
```

For two outcomes both priced at 2.10, `S` is approximately 0.952381. A total stake of EUR 100 splits into EUR 50 on each outcome. Either successful leg returns EUR 105, producing EUR 5 profit and a five-percent ROI before unmodeled costs or execution changes.

For three outcomes priced at 3.60, 3.60, and 3.60, `S` is approximately 0.833333 and the ideal ROI is twenty percent. That unusually high value is a reason to inspect matching and freshness carefully. A large displayed return can signal a mapping mistake rather than a remarkable real opportunity.

9.1 Rounding matters

Bookmakers do not generally accept an infinitely precise stake. The engine uses monetary increments and recalculates the payout for every settlement outcome after rounding. The worst-case payout is the smallest resulting payout, and worst-case profit is that payout minus the actual sum staked. A theoretically positive result can disappear when stakes are rounded, especially for a small bankroll.

9.2 Conditions the simple calculation does not cover

The formula assumes a complete set of outcomes with the stated winning return and no additional deductions. It does not automatically model fees, taxes, exchange commission, changing currency conversion, pushes, quarter-line Asian settlements, dead heats, free-bet rules, bonuses, or different void conditions. Do not compare those contracts as if they were simple two-way or three-way bets.

The same-named market is not always the same contract. Match winner with overtime is different from regulation-time winner; qualification is different from match result; a first-set market is different from the whole match. Settlement rules must be explicit and compatible before prices are compared.

10. Freshness, matching, and coverage

The scanner stores when it observed a quote. When a source supplies an actual quote-update time, that is a separate fact. A newly downloaded cached page can contain an old price; retrieval time alone cannot prove the bookmaker just updated it. By default, the scanner requires source-update timestamps and rejects quotes without them. If you explicitly disable that requirement to use an otherwise suitable public page, each resulting opportunity warns that actual price age is unknown. Check the current offered price before execution.

Defaults are a 0.5-percent minimum profit, EUR 100 stake budget, 120-second maximum age, and a 60-second quote-time spread. Adjust these deliberately; increasing the age limit allows older prices and does not refresh them.

Event matching uses exact sport, league, ordered home and away names, and scheduled start instant after case and whitespace normalization. It also requires identical period, rules, line, and currency. There is no fuzzy name matching or automatic alias conversion. Home and away labels must retain their meaning. A reserve team, women's team, youth team, esports participant, or similarly named club is a separate participant. Conservative matching can miss some opportunities; uncertain events should not be merged merely to increase the count.

Broad coverage requires maintenance. Bookmakers add, remove, suspend, and rename markets, and scraper layouts change. Add sports and leagues only when the data includes enough information to preserve event and settlement identity.

11. Imports and reproducible testing

The dashboard's quote import workflow lets you analyze data you obtained legitimately or test a known example. Use the canonical quote format described in Code Explained. Imported quotes are not automatically live or verified. Their timestamps must reflect the real observations, and their source label should explain where they came from.

Use imports for debugging and reproducing a discrepancy. Remove any personal or account data from a sample before sharing it. Preserve the original market period, settlement rules, participants, odds, and timestamps so a bug report can actually be reproduced.

Run the test suite from the checkout root:

```
uv run pytest
```

Tests can verify math, matching, parsing, HTTP behavior, storage, and API contracts with controlled data. They do not demonstrate that a bookmaker's live public page is reachable in your region today. Keep those two forms of validation separate.

12. Troubleshooting

12.1 The startup command is not found

Open a new terminal after installing uv. Confirm `uv --version` works. Change into the directory containing `pyproject.toml`, then rerun the frozen sync. If you are using a downloaded archive, make sure you did not stay in its parent directory.

12.2 Dependency installation fails

Read the first meaningful package error. Check the supported Python version, network access, disk space, and certificate configuration. In a corporate or managed network, preserve its configured trust chain. Never fix a package error by disabling TLS or replacing expected artifact hashes.

12.3 The page does not open

Keep the server terminal open and check for a traceback or port conflict. Open the exact address printed by the server. Another device cannot normally use a service bound to loopback. Do not expose it publicly just to make a local setup work.

12.4 A source is unconfigured

Open that source's settings and verify the URL, mode, mapping, constants, required rule confirmation, and enabled state. A provisional default domain is not a completed scraper. Use its separate setup PDF.

12.5 A source is blocked

Inspect the status message. A robots denial, access challenge, login requirement, geoblock, or HTTP denial is a source access limitation. Test public access in an anonymous browser on the intended machine. Do not paste cookies or add evasion tools to force the connector through an access control.

12.6 A source returns zero quotes

Check whether the exact target page has future events and the requested market. Re-inspect event and quote selectors, field attributes, JSON paths, time formats, decimal odds, and suspended-state indicators. Confirm the mapping extracts the event's complete outcome set. A missing field should be corrected in the mapping rather than replaced with a guessed value.

12.7 The results contain implausible profit

Check event identity first. Then check home/away orientation, period, overtime, line, market type, decimal-versus-fractional odds, start time, source age, and outcome completeness. Disable the suspect source until you can reproduce and correct the issue. Do not place bets based on an unexplained extreme return.

12.8 No opportunities appear

Verify that at least two bookmakers produced compatible current markets. Lower the minimum-profit threshold only if you understand its effect. Check the freshness and quote-time spread limits, future start times, filters, line precision, rules, and supported market types. The prices may simply offer no arbitrage.

12.9 Browser mode fails

Confirm the optional browser extra and Chromium are installed in the same environment used to run the app. Check system library errors on Linux. Use a realistic wait selector for the public odds container. If the page shows a challenge or login requirement, the problem is access, not a missing selector.

12.10 API key or quota errors

Save the key again using the masked dashboard field and check your provider account. Verify its plan covers the requested endpoint, sport, region, and market. Reduce request scope if the allowance is small. Do not include the key in a screenshot or error report.

13. Maintenance, backups, and updates

Stop the server before copying its local database and credential material for a consistent backup. The default `.data` folder is ignored by Git; for storage outside the checkout, use `uv run arbscanner --data-dir C:\Users\YourName\ArbitrageFinderData` on Windows or the corresponding absolute path on your system. The `ARBSCANNER_DATA_DIR` environment variable is also supported. A backup containing both encrypted credentials and their decryption material must itself be treated as sensitive. Deleting a key in the dashboard does not erase copies in old backups.

Before updating code, preserve your configuration and inspect your Git changes. Pull the new code using your normal workflow, run `uv sync --frozen`, run the test suite, and retest demo mode plus one narrow real-source scan. Review source mappings whenever a bookmaker changes its page.

The application's scan history is useful for diagnosing a past result; it is not a guarantee that the quoted price was accepted. Keep your actual bets and settlement records separately if you need accounting.

14. Deployment and zero-cost boundaries

The supported baseline is one person running the application on their own computer. The local server does not include the account management, hardened public authentication, TLS termination, tenant isolation, or operational controls expected of a public betting service. Keep it private and use loopback binding.

Running on a VPS usually introduces hosting cost and may change bookmaker accessibility. Free cloud tiers can sleep, limit requests, forbid scraping, or change their terms. The application does not depend on one. A cloud environment that can run the demo may still be unable to fetch bookmaker pages because of regional or outbound restrictions.

Do not publish a dashboard containing your API keys or source configuration. If you later need multi-user or Internet-facing access, treat that as a separate deployment project with authentication, HTTPS, secret management, access logging, backups, and source licensing reviewed for that use.

15. Add another bookmaker

1. Verify public anonymous access and the target page's applicable rules.
2. Identify whether HTML, public JSON, browser rendering, or a documented provider feed supplies the data.
3. Map a small, known event and one supported market to the canonical quote format.
4. Use the custom scraper support or implement a dedicated connector through the extension points in Code Explained.
5. Add parsing fixtures and tests for event identity, suspended odds, incomplete outcomes, and malformed prices.
6. Test one real request on the intended machine and record the verification date, domain, mode, market, and limitations.
7. Enable the source only after comparing normalized results against its public display.

16. Rebuild these PDFs

The PDF files are generated artifacts from the Markdown in `docs/`. To rebuild all seven manuals:

```
uv sync --frozen --group docs
uv run --group docs python tools/build_docs.py
```

The generator uses ReportLab locally and makes no network request. The PDFs have a linked contents page and navigation bookmarks. If you change application behavior, update the documentation source before rebuilding; a successful PDF build does not establish that the written instructions are accurate.

17. Daily operating checklist

1. Start the local server and confirm your intended currency and bankroll.
2. Confirm live mode before analyzing real sources, or demo mode when practicing.
3. Review enabled source status and keep only configured sources active.
4. Set minimum profit, freshness, quote-time spread, and filters.
5. Press Scan once and wait for completion.
6. Inspect source status, quote count, market rules, timestamps, and any warnings.
7. Independently verify every leg before any manual bet.
8. Stop the server when you finish and keep local credentials private.